فصل ۴: درخت‌ها (BST و AVL) و هیپ

سهم در کنکور: ۷ سؤال در ارشد ۱۳۹۸ تا ۱۴۰۴ (۴ سؤال درخت و ۳ سؤال هیپ). تیپ‌ها: «چه پرسمانی بدون اطلاعات اضافه در `O(log n)` جواب دارد؟»، کران ارتفاع، تعداد چرخش، و جای اعداد در هیپ. **زمان مطالعه:** دو بلوک ۹۰ دقیقه‌ای (درس ۱ تا ۳، بعد درس ۴ و ۵).

درس ۱: درخت دودویی و پیمایش‌ها

- **ارتفاع** = تعداد یال‌های بلندترین مسیر از ریشه تا برگ. درخت با ارتفاع h حداکثر `2^(h+1) - 1` گره دارد، پس $h \geq \lfloor \log_2 n \rfloor$.
- در هر درخت دودویی: **تعداد برگ‌ها = تعداد گره‌های دو فرزندی + ۱**.
- تعداد درخت‌های دودویی متمایز با n گره، عدد کاتالان C_n است.
- **درخت کامل (complete):** همهٔ سطح‌ها پر، جز احتمالاً سطح آخر که از چپ پر می‌شود. شکل هیپ همین است.

پیمایش	ترتیب
پیش‌ترتیب	ریشه، چپ، راست
میان‌ترتیب	چپ، ریشه، راست (در BST خروجی **مرتب** است)
پس‌ترتیب	چپ، راست، ریشه

بازسازی: «پیش‌ترتیب + میان‌ترتیب» یا «پس‌ترتیب + میان‌ترتیب» درخت را **یکتا** تعیین می‌کنند. «پیش + پس» یکتا نیست (مگر درخت پر باشد).

روش بازسازی: اولین عضو پیش‌ترتیب ریشه است. جای آن در میان‌ترتیب، زیردرخت چپ و راست را جدا می‌کند. همین کار را بازگشتی تکرار کن.

درس ۲: درخت جستجوی دودویی (BST)

- قاعده: همهٔ کلیدهای زیردرخت چپ < ریشه < همهٔ کلیدهای زیردرخت راست.
- جستجو، درج، حذف، min، max، پیش‌رو و پس‌رو همه **`O(h)`** هستند.
- **حذف گرهٔ دوفرزندی:** کلیدش را با **پس‌رو** (کوچک‌ترین کلید زیردرخت راست) یا پیش‌رو عوض کن.
- **h بین `log n` و `n` است.** درج کلیدها به‌ترتیب مرتب، درخت را یک **زنجیر** با ارتفاع `n - 1` می‌کند.

پرسمان‌هایی که اطلاعات اضافه می‌خواهند

در BST معمولی (هر گره فقط کلید و دو اشاره‌گر):

پرسمان	؟ O(h)
کوچک‌ترین یا بزرگ‌ترین	✅ (همیشه چپ یا همیشه راست)
آیا x هست؟	✅
رتبهٔ x (چندمین کوچک‌ترین است)	❌ به **اندازهٔ زیردرخت** در هر گره نیاز دارد
میانه یا kامین کوچک‌ترین	❌ همان دلیل

کران ارتفاع از روی جمع هزینه‌ها

اگر یک مسیر به طول h وجود داشته باشد، گره‌های روی آن هزینه‌های `1, 2, …, h` دارند و جمعشان حدود `h²/2` است. پس اگر جمع کل هزینه‌ها S باشد، `h = O(√S)` .

درخت وزن‌متوازن

اگر در هر گره، اندازهٔ هر زیردرخت حداکثر کسر c از اندازهٔ گره باشد (`c < 1`)، ارتفاع حداکثر `log_(1/c) n` است. مثلاً با `c = 2/3` ارتفاع `log_(3/2) n ≈ 1.71 log₂ n` است.

درس ۳: AVL و چرخش

- **AVL:** در هر گره، اختلاف ارتفاع زیردرخت چپ و راست حداکثر ۱ است. ارتفاع `≈ 1.44 log₂ n` .
- **حداقل گره در AVL با ارتفاع h:** `N(h) = N(h−1) + N(h−2) + 1` ، با مقادیر `N(0)=1، N(1)=2، N(2)=4، N(3)=7، N(4)=12، N(5)=20`.
- **چرخش‌ها:** حالت‌های LL و RR با یک چرخش ساده درست می‌شوند؛ LR و RL با چرخش دوگانه. **درج** در AVL حداکثر یک چرخش (ساده یا دوگانه) لازم دارد. **حذف** ممکن است تا `O(log n)` چرخش بخواهد.

ترفند «ستون راست» برای شمردن حداقل چرخش

ستون راست یعنی مسیری که از ریشه فقط به فرزند راست می‌رود. چرخش چپ روی یک گرهٔ این ستون، طولش را **دقیقاً یکی** کم می‌کند و چرخش‌های دیگر حداکثر یکی تغییرش می‌دهند. پس: **حداقل چرخش ≥ (طول ستون راست اول) − (طول ستون راست هدف).**

درس ۴: هیپ (Heap)

- درخت **کامل** که در هر گره، کلید والد ≤ کلید فرزندان است (هیپ بیشینه). در آرایه با اندیس از ۱: والد `⌊i/2⌋` و فرزندان `2i` و `2i+1`.

عمل	زمان
یافتن بیشینه	`O(1)`
درج (بالا بردن)	`O(log n)`
حذف بیشینه (آخرین عضو را در ریشه بگذار و پایین ببر)	`O(log n)`
افزایش کلید	`O(log n)`
ساخت هیپ از n عدد (build-heap)	**`O(n)`**، نه `O(n log n)`
جستجوی یک کلید دلخواه	`O(n)` (هیپ برای جستجو نیست)
کمینه در هیپ بیشینه	بین برگ‌ها، یعنی `⌈n/2⌉` مقایسه

نکته‌های تستی

- kامین بزرگ‌ترین عضو در عمق حداکثر `k − 1` است. مثلاً دومین بزرگ‌ترین حتماً فرزند ریشه است.
- مجموعهٔ گره‌هایی که کلیدشان از یک مقدار بزرگ‌تر است، **یک زیردرخت همبند شامل ریشه** است (چون والد هر کدام هم بزرگ‌تر است).
- برای اینکه t برگ در عمق d درون چنین زیردرختی باشند، حداقل `Σ ⌈t/2^(d−i)⌉` (روی همهٔ سطح‌های i) گره لازم است.
- **کجا هیپ لازم است:** پریم، دایکسترا، هافمن (صف اولویت). **کراسکال** به مرتب‌سازی یال‌ها و Union-Find نیاز دارد، نه هیپ.
- **هیپ dتایی:** ارتفاع `log_d n`. درج سریع‌تر است، ولی حذف بیشینه `O(d·log_d n)` هزینه دارد.

درس ۵: انتخاب ساختار ترکیبی

اگر چند عمل با هم خواسته شده، برای هر عمل ببین کدام ساختار جوابش را می‌دهد و ترکیبشان کن:

- **درخت متوازن (AVL یا قرمز-سیاه)** با اندازهٔ زیردرخت: درج، حذف، min و max، میانه، رتبه، همه `O(log n)`.
- **جدول درهم‌سازی:** پیدا کردن عنصر با کلید در امید `O(1)`.
- **هیپ:** فقط بیشینه یا کمینه سریع است، میانه نه.

تمرین فصل ۴

۱. پیش‌ترتیب `A B D E C F` و میان‌ترتیب `D B E A F C` است. پس‌ترتیب چیست؟

۲. حداکثر گره در درخت دودویی با ارتفاع ۴؟ حداقل گره در AVL با ارتفاع ۴؟

۳. کلیدهای ۳۰، ۲۰ و ۱۰ را به‌ترتیب در AVL درج کن. ریشهٔ نهایی؟

۴. هیپ بیشینهٔ `[50, 30, 40, 10, 20, 35]`. بعد از حذف بیشینه آرایه چه می‌شود؟

۵. در هیپ بیشینه با کلیدهای متمایز، سومین بزرگ‌ترین عضو در چه عمق‌هایی ممکن است باشد؟

> 🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۵: انتخاب ساختار ترکیبی)

📝 آزمون فصل ۴ (سؤال‌های واقعی کنکور ارشد کامپیوتر)

زمان: ۲۰ دقیقه برای ۷ سؤال.

۱. (ارشد ۱۴۰۱، سؤال ۶۷) فرض کنید یک درخت دودویی جستجو بر روی n عدد حقیقی متمایز با ارتفاع `O(log n)` در اختیار داریم. چه تعداد از پرسمان‌های زیر را بدون پیش‌پردازش و اطلاعات اضافی می‌توان در `O(log n)` پاسخ داد؟ (در هر گره صرفاً یک کلید و دو اشاره‌گر به فرزندان نگه‌داشته شده است.)

- محاسبهٔ کوچک‌ترین عدد
- محاسبهٔ میانه
- تعیین آنکه آیا عدد داده‌شدهٔ x در درخت وجود دارد
- محاسبهٔ مرتبهٔ عدد داده‌شدهٔ x در بین n عدد ذخیره‌شده در درخت

۱) صفر ۲) ۳ ۳) ۲ ۴) ۱

۲. (ارشد ۱۴۰۰، سؤال ۶۲) اعداد `1, 2, …, 10` را به‌ترتیب از چپ به راست در یک درخت دودویی جستجو که در اول کار تهی است درج می‌کنیم. بعد از درج همهٔ عناصر می‌خواهیم درخت حاصل را به درخت دودویی جستجو با ارتفاع ۳ تبدیل کنیم. برای این کار تنها مجاز به استفاده از عمل چرخش (به چپ یا راست) هستیم. با حداقل چندبار چرخش می‌توان این کار را انجام داد؟ (منظور از چرخش همان عملیات متعارفی است که برای متوازن‌سازی درخت‌های دودویی جستجو استفاده می‌شود.) ۱) ۶ ۲) ۷ ۳) ۸ ۴) ۹

۳. (ارشد ۱۳۹۹، سؤال ۸۸) اعداد صحیح $x_1, \ldots, x_n$ را در یک درخت دودویی جستجو با ارتفاع h ذخیره کرده‌ایم. فرض کنید هزینهٔ جستجوی x_i (تعداد مقایسه‌های لازم در درخت برای پیدا کردن x_i) برابر c_i باشد. می‌دانیم $\sum c_i = O(n \log n)$ است. کدام گزینه درست است؟ ۱) $h = \Omega(\sqrt{n})$ ۲) $h = O(\log n)$ ۳) $h = O(\sqrt{(n \log n)})$ ۴) می‌توان مثالی زد که $h = \Omega(n)$ باشد.

۴. (ارشد ۱۳۹۸، سؤال ۵۲) فرض کنید یک درخت دودویی جستجو با n گره داریم. به ازای گرهٔ u از این درخت، وزن آن را تعداد گره‌ها در زیردرخت به ریشهٔ u (شامل u) در نظر بگیرید. می‌دانیم در درخت فوق به ازای هر گرهٔ داخلی u، نسبت وزن فرزند چپ و فرزند راست حداقل ۰٫۵ و حداکثر ۲ است. بهترین کران بالا برای زمان جستجو در این

درخت در بدترین حالت، در بین گزینه‌ها کدام است؟ (مبنای لگاریتم‌ها ۲ است.) ۱) $1.5 \log n$ ۲) $2.5 \log n$ ۳) n ۴) $2 \log n$ ۴) $3 \log n$

۵. (ارشد ۱۴۰۴، سؤال ۶۵) فرض کنید یک ساختار دادهٔ جدید به نام لیست اولویت‌دار پویا (Dynamic Priority List) دارید. این ساختار داده از لیستی از گره‌ها تشکیل شده است که هر گره دارای یک مقدار کلید و یک اولویت است. عمل‌های زیر بر روی این ساختار داده تعریف شده‌اند:

- `insert(x, p)` : افزودن یک عنصر با کلید x و اولویت p به لیست
- `DeleteMax()` : حذف گره‌ای که دارای بیشترین اولویت است. اگر چند گره دارای اولویت یکسان باشند، اولین گره از سمت چپ حذف می‌شود.
- `FindMedian()` : یافتن مقدار میانه از بین تمامی کلیدهای موجود در لیست
- `IncreasePriority(x, k)` : افزایش اولویت عنصر با کلید x به اندازهٔ k

هرکدام از این عملیات باید در کمترین پیچیدگی زمانی ممکن انجام شوند. حال، کدام‌یک از روش‌های زیر مناسب‌ترین ساختار دادهٔ ترکیبی برای پیاده‌سازی لیست اولویت‌دار پویا است، به گونه‌ای که همهٔ عملیات با پیچیدگی بهینه انجام شوند؟

۱. استفاده از یک پشته و یک صف به‌صورت هم‌زمان برای مدیریت ترتیب ورود و خروج عناصر و اولویت آن‌ها

۲. ترکیب یک درخت فیبوناچی (Fibonacci Heap) و یک لیست پیوندی ساده برای مدیریت عناصر و اولویت‌ها

۳. ترکیب یک هرم دوتایی (Double-ended Heap) برای مدیریت اولویت‌ها و یک ساختار دادهٔ مرتب برای دسترسی سریع به میانه

۴. ترکیب یک درخت جستجوی دودویی متوازن (Balanced Binary Search Tree) برای ذخیرهٔ اولویت‌ها و یک جدول درهم‌سازی (Hash Table) برای دسترسی سریع به عناصر

۶. (ارشد ۱۴۰۲، سؤال ۶۱) برای پیاده‌سازی مؤثر کدام‌یک از الگوریتم‌های زیر، ساختمان دادهٔ heap لازم **نیست**؟ ۱) پریم ۲) دایکسترا ۳) هافمن ۴) کراسکال

۷. (ارشد ۱۴۰۱، سؤال ۶۴) اعداد یک تا ۱۲۷ در یک هرم بیشینه که به‌صورت یک درخت دودویی کامل با ارتفاع ۶ پیاده‌سازی شده، قرار گرفته‌اند. حداکثر تعداد برگ با مقدار بیشتر از ۱۰۰ در این درخت چقدر می‌تواند باشد؟ ۱) ۲۰ ۲) ۱۲ ۳) ۱۱ ۴) ۲

> 🔑 **جواب‌ها و راه‌حل (کلید رسمی سازمان سنجش):** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (آزمون فصل ۴ (سؤال‌های واقعی کنکور ارشد کامپیوتر))

🗂 کارت خلاصهٔ فصل ۴

- برگ‌ها = گره‌های دوفرزندی + ۱. حداکثر گره `2^(h+1) - 1` . پیش یا پس‌ترتیب + میان‌ترتیب یعنی بازسازی یکتا.

- BST: همهٔ عمل‌ها `O(h)` . درج مرتب یعنی زنجیر. میانه و رتبه به اندازهٔ زیردرخت نیاز دارند.
- `S = هزینه Σ` ⇐ `h = O(√S)` . وزن‌متوازن با کسر c ⇐ `h ≤ log_(1/c) n` .
- AVL: `N(h) = N(h−1) + N(h−2) + 1` (`1, 2, 4, 7, 12, 20`). درج حداکثر یک چرخش دارد. حداقل چرخش ≤ اختلاف طول ستون راست.
- هیپ: ساخت `O(n)` ؛ درج و حذف `O(log n)` ؛ جستجو `O(n)` ؛ کمینه بین برگ‌ها؛ kام بزرگ در عمق ≤ `k−1`؛ گره‌های «بزرگ‌تر از x» زیردرخت همبند شامل ریشه‌اند.
- کراسکال هیپ نمی‌خواهد. میانه + اولویت + دسترسی با کلید: BST متوازن + جدول درهم‌سازی.

🔑 پاسخ‌نامهٔ فصل

بخش ۱: درس ۵: انتخاب ساختار ترکیبی

۱. `D E B F C A`

۲. $2^5 - 1 = 31$ و $N(4) = 12$

۳. حالت LL است و یک چرخش راست لازم دارد. ریشهٔ نهایی **۲۰** است، با فرزندان ۱۰ و ۳۰.

۴. ۳۵ به ریشه می‌رود: `[35, 30, 40, 10, 20]`. بعد با ۴۰ جابه‌جا می‌شود: `[40, 30, 35, 10, 20]`.

۵. عمق ۱ یا ۲.

بخش ۲: آزمون فصل ۴ (سؤال‌های واقعی کنکور ارشد کامپیوتر)

سؤال	۱	۲	۳	۴	۵	۶	۷
کلید	۳	۱	۳	۳	۴	۴	۲

۱. گزینهٔ ۳ (دو تا). کوچک‌ترین عدد و جستجو در $O(h)$ جواب دارند. میانه و رتبه به اندازهٔ زیردرخت در گره‌ها نیاز دارند.

۲. گزینهٔ ۱ (۶).

- **کران پایین:** درخت اول زنجیر راست است و ستون راستش ۱۰ گره دارد. درخت با ارتفاع ۳ حداکثر ۴ گره روی ستون راست دارد. هر چرخش طول ستون راست را حداکثر یکی تغییر می‌دهد، پس حداقل $10 - 4 = 6$ چرخش لازم است.
- **ساخت با ۶ چرخش:** چرخش چپ روی ۱، بعد ۳، بعد ۲ زیردرخت $2(1,3)$ را زیر ۴ می‌سازد. چرخش چپ روی ۵، بعد ۷، بعد ۶ زیردرخت $6(5,7)$ را زیر ۸ می‌سازد. درخت نهایی: ریشه ۴، چپ $2(1,3)$، راست 8 با چپ $6(5,7)$ و راست $9 \to 10$. ارتفاعش ۳ است.

۳. گزینهٔ ۳. مسیر به طول h هزینه‌ای حدود $h^2/2$ دارد که باید $\Sigma c_i = O(n \log n) \le$ باشد. پس $h = O(\sqrt{(n \log n)})$.

- گزینهٔ ۱ غلط است: درخت متوازن $h = \log n$ دارد.
- گزینهٔ ۲ غلط است: یک مسیر به طول $\sqrt{(n \log n)}$ کنار یک درخت متوازن، جمع را $O(n \log n)$ نگه می‌دارد.
- گزینهٔ ۴ غلط است: با $h = \Omega(n)$ جمع $\Omega(n^2)$ می‌شد.

۴. گزینهٔ ۳. با نسبت ۰٫۵ تا ۲ بین دو فرزند، هر فرزند حداکثر $2/3$ وزن والد را دارد. پس عمق حداکثر $\log_{(3/2)} n \approx 1.71 \log n$ است. $1.5 \log n$ کران درستی نیست، و بین بقیه کوچک‌ترین کران درست

$2 \log n$ است.

۵. گزینهٔ ۴. درخت متوازن (با اندازهٔ زیردرخت) درج، حذف بیشینه، میانه و تغییر اولویت را در $O(\log n)$ انجام می‌دهد، و جدول درهم‌سازی عنصر x را در امید $O(1)$ پیدا می‌کند. هیپ میانه را نمی‌دهد و پشته و صف اولویت را مدیریت نمی‌کنند.

۶. گزینهٔ ۴. کراسکال یال‌ها را مرتب می‌کند و Union-Find می‌خواهد. پریم و دایکسترا و هافمن صف اولویت (هیپ) لازم دارند.

۷. گزینهٔ ۲ (۱۲). اعداد بزرگ‌تر از ۱۰۰ (یعنی ۲۷ عدد، از ۱۰۱ تا ۱۲۷) یک زیردرخت همبند شامل ریشه می‌سازند. برای t برگ در عمق ۶، در هر عمق d حداقل $\lceil t/2^{(6-d)} \rceil$ گره لازم است:

- برای $t = 12$: $12 + 6 + 3 + 2 + 1 + 1 + 1 = 26 \le 27$ ✓
- برای $t = 13$: $13 + 7 + 4 + 2 + 1 + 1 + 1 = 29 > 27$ ✗

پس حداکثر **۱۲** است.